



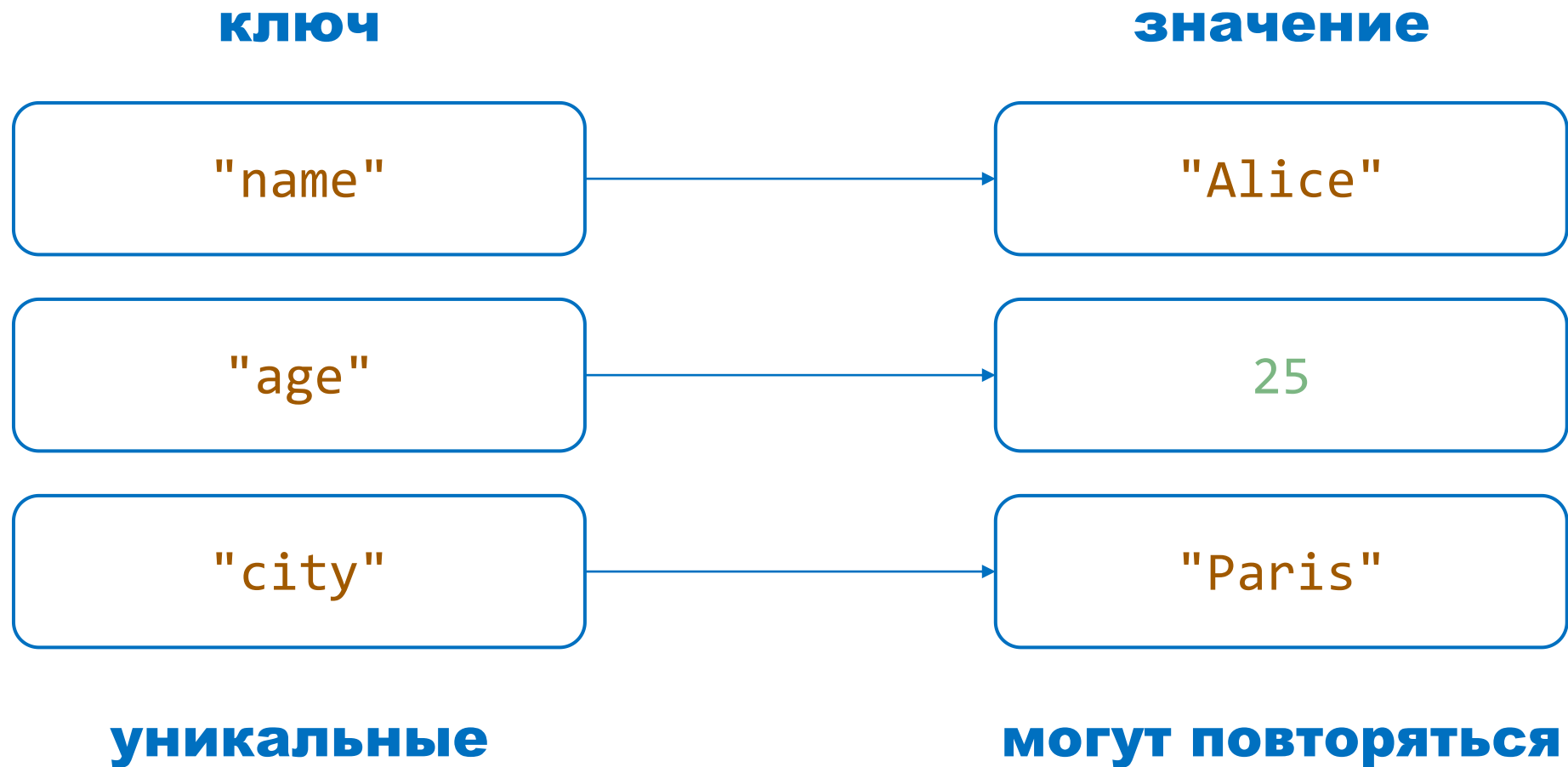
Словари и множества

Докладчик: Евграфов Михаил



Словарь: обзор

Ключ и значение



Характеристики словаря

гетерогенный

ограниченный

упорядоченный

изменяемый

итерируемый

Гетерогенность

```
mixed_dict = {  
    "name": "Alice",           # ключ-строка, значение-строка  
    "scores": [90, 85],       # значение-список  
    42: "answer",             # ключ-число  
    (1, 2): {"nested": True}, # ключ-кортеж, значение-словарь  
}
```

Ограниченность

```
user = {  
    "name": "Alice",  
    "age": 25,  
    "city": "Paris",  
}  
  
print(len(user))    # 3  
print(len({}))      # 0
```

Упорядоченность

```
ordered_dict = {}
```

```
ordered_dict["first"] = 1
```

```
ordered_dict["second"] = 2
```

```
ordered_dict["third"] = 3
```

```
print(ordered_dict)
```

```
# {'first': 1, 'second': 2, 'third': 3}
```

Равенство словарей

```
first_dict = {"a": 1, "b": 2}  
second_dict = {"b": 2, "a": 1}
```

```
print(first_dict == second_dict)  
# True - порядок пар не влияет на равенство  
print([1, 2] == [2, 1])  
# False - для списков порядок важен
```

Изменяемость

```
scores = {"math": 90}  
identity_before = id(scores)
```

```
scores["physics"] = 85  
scores["math"] = 95
```

```
print(scores)  
# {'math': 95, 'physics': 85}  
print(id(scores) == identity_before)  
# True - это все еще тот же объект
```

Итерируемость

```
user = {"name": "Alice", "age": 25}
```

```
for key in user:  
    print(key)
```

```
# name
```

```
# age
```

Где применяются словари

описание объекта

**таблицы
соответствий**

**подсчет и
группировка**

**доступ по
идентификатору**

JSON

Словари внутри Python

`globals()`

**глобальные
переменные**

`__dict__`

атрибуты объектов

`**kwargs`

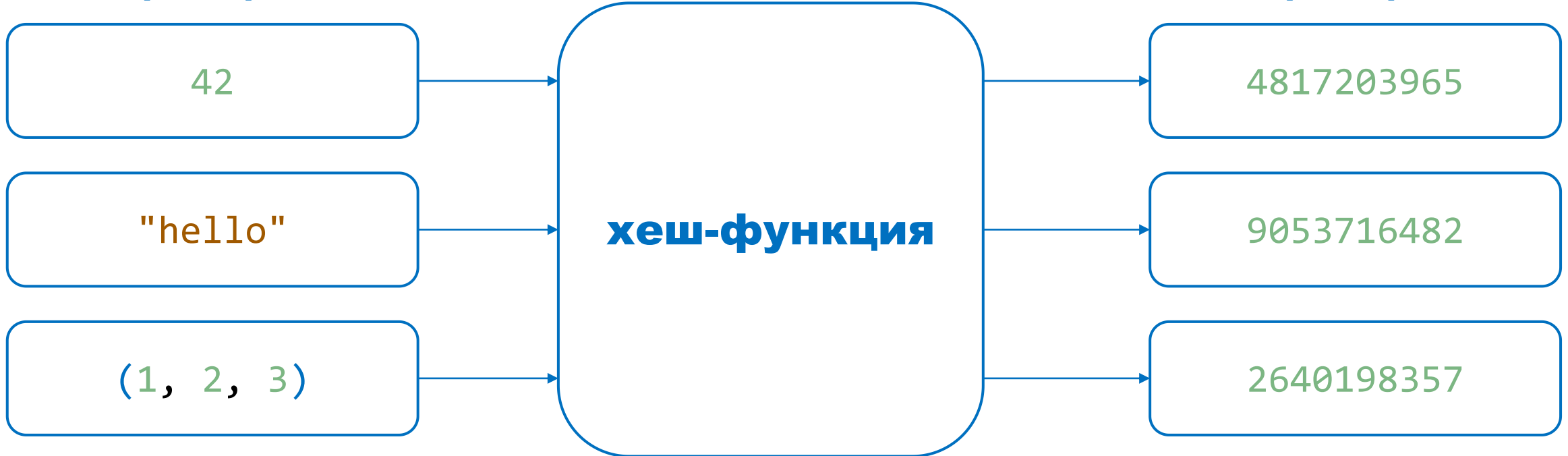
**именованные
аргументы**



Хешируемость

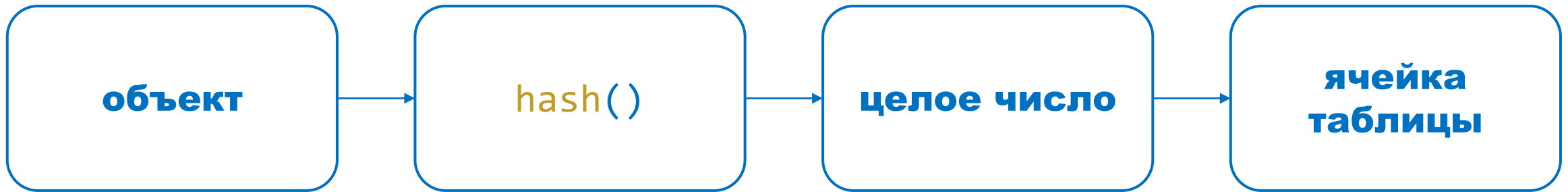
Хеш-функция

объекты разного
размера



один и тот же объект — всегда один и тот же хеш

Хеш-таблица



место находим сразу, без перебора

Функция `hash()`

```
print(hash(42))           # 42
print(hash(True))         # 1
print(hash("hello"))      # некоторое целое число

print(hash(3) == hash(3.0))
# True
```

Хешируемые объекты

```
print(hash(42))  
print(hash(3.14))  
print(hash(True))  
print(hash("hello"))  
print(hash(None))  
print(hash(frozenset({1, 2})))  
print(hash((1, 2, "a")))
```

Нехешируемые объекты

```
print(hash([1, 2, 3]))  
# TypeError: unhashable type: 'list'
```

```
print(hash({"a": 1}))  
# TypeError: unhashable type: 'dict'
```

```
print(hash({1, 2, 3}))  
# TypeError: unhashable type: 'set'
```

```
print(hash((1, [2, 3])))  
# TypeError: unhashable type: 'list'
```

Хеш кортежа

```
print(hash((1, 2, 3)))  
# хеш успешно вычислен
```

```
print(hash((1, [2, 3])))  
# TypeError: unhashable type: 'list'
```

Хешируемость

хешируемые

```
int  
float  
bool  
str  
None  
frozenset  
(1, 2, "a")
```

нехешируемые

```
list  
dict  
set  
(1, [2, 3])
```



Создание словарей

Способы создания

`{key: value}`
литерал

`{key: value for ...}`
включение

`dict(name=value)`
конструктор

`dict.fromkeys(keys)`
метод класса

Словарные литералы

```
empty_dict = {}  
user = {"name": "Alice", "age": 25}  
square_roots = {1: 1.0, 4: 2.0, 9: 3.0}  
  
print(empty_dict)      # {}  
print(user)            # {'name': 'Alice', 'age': 25}  
print(square_roots)    # {1: 1.0, 4: 2.0, 9: 3.0}
```

Многострочный литерал

```
config = {  
    "host": "localhost",  
    "port": 8080,  
    "debug": True,  
}
```

Ключи хешируемые

```
valid_dict = {(1, 2): "point"}  
print(valid_dict[(1, 2)])  
# point
```

```
invalid_dict = {[1, 2]: "point"}  
# TypeError: unhashable type: 'list'
```

Повторяющиеся ключи

```
duplicate_dict = {"a": 1, "b": 2, "a": 3}
```

```
print(duplicate_dict)
```

```
# {'a': 3, 'b': 2}
```

```
print({1: "int", 1.0: "float", True: "bool"})
```

```
# {1: 'bool'}
```

Словарное включение

{ключ: значение for переменная in источник}

```
squares = {  
    number: number ** 2  
    for number in range(1, 6)  
}  
  
print(squares)  
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Включение с условием

{ключ: значение **for** переменная **in** источник **if** условие}

```
even_squares = {  
    number: number ** 2  
    for number in range(1, 11)  
    if number % 2 == 0  
}
```

```
print(even_squares)
```

```
# {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

dict()

```
empty_dict = dict()  
print(empty_dict)  
# {}
```

```
user = dict(name="Alice", age=25)  
print(user)  
# {'name': 'Alice', 'age': 25}
```

dict(): другой словарь

```
original = {"a": 1, "b": 2}  
copied = dict(original)
```

```
copied["c"] = 3
```

```
print(original)    # {'a': 1, 'b': 2}  
print(copied)      # {'a': 1, 'b': 2, 'c': 3}
```

dict(): пары

```
pairs = [("name", "Alice"), ("age", 25)]  
print(dict(pairs))  
# {'name': 'Alice', 'age': 25}
```

```
print(dict([("a", 1, 2)]))  
# ValueError: ...
```

dict(): комбинация

```
base = {"a": 1, "b": 2}
```

```
print(dict(base, c=3))  
# {'a': 1, 'b': 2, 'c': 3}
```

```
print(dict(base, a=100))  
# {'a': 100, 'b': 2}
```

`dict.fromkeys()`

```
keys = ["a", "b", "c"]
```

```
print(dict.fromkeys(keys))
```

```
# {'a': None, 'b': None, 'c': None}
```

```
print(dict.fromkeys(keys, 0))
```

```
# {'a': 0, 'b': 0, 'c': 0}
```

fromkeys и список

```
shared_dict = dict.fromkeys(["a", "b", "c"], [])  
shared_dict["a"].append(1)
```

```
print(shared_dict)  
# {'a': [1], 'b': [1], 'c': [1]}
```

```
print(shared_dict["a"] is shared_dict["b"])  
# True - это один и тот же список
```

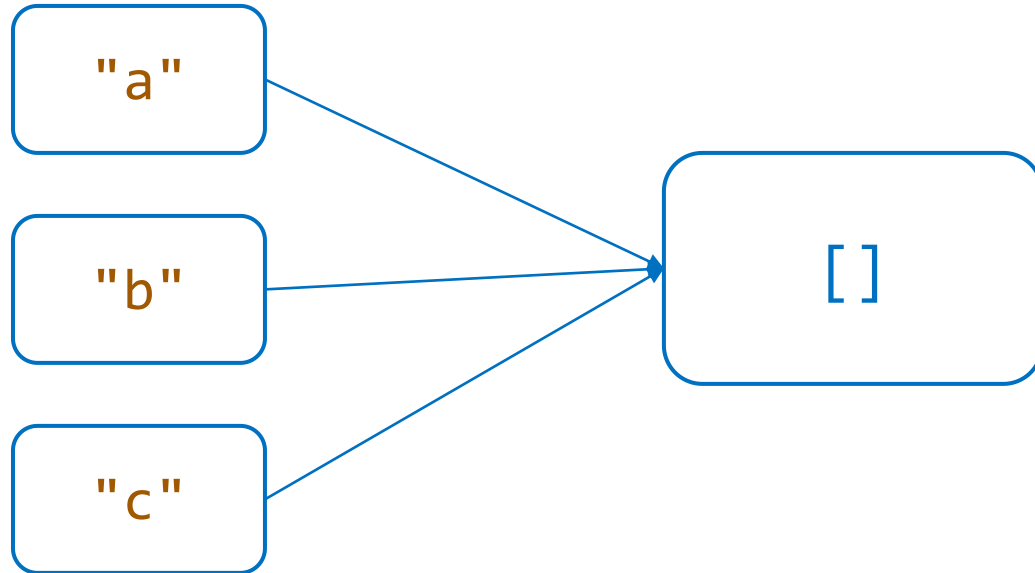
Отдельные списки

```
separate_dict = {key: [] for key in ["a", "b", "c"]}
separate_dict["a"].append(1)
```

```
print(separate_dict)
# {'a': [1], 'b': [], 'c': []}
```

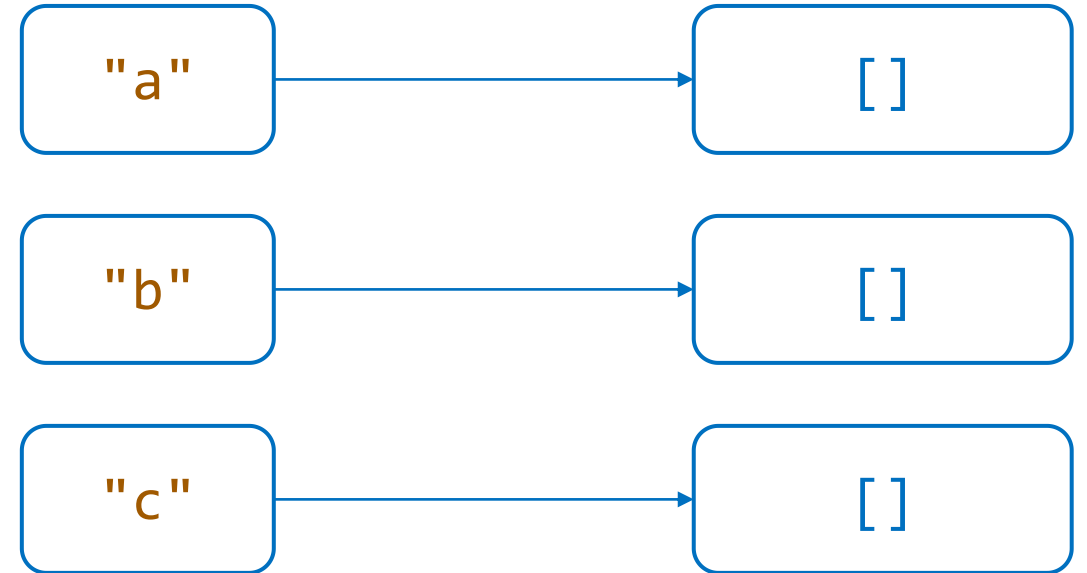
fromkeys и список

```
dict.fromkeys(keys, [])
```



один и тот же список

```
{key: [] for key in keys}
```



отдельные списки



Итерирование

Обход ключей

```
prices = {"apple": 100, "banana": 60, "cherry": 250}
```

```
for name in prices:  
    print(name)
```

```
# apple  
# banana  
# cherry
```

keys, values, items

`prices.keys()`

ключи

`prices.values()`

значения

`prices.items()`

пары (ключ, значение)

Представления

```
prices = {"apple": 100, "banana": 60}
```

```
print(prices.keys())  
# dict_keys(['apple', 'banana'])
```

```
print(prices.values())  
# dict_values([100, 60])
```

```
print(prices.items())  
# dict_items([('apple', 100), ('banana', 60)])
```

Обход значений

```
prices = {"apple": 100, "banana": 60}
```

```
for price in prices.values():  
    print(price)
```

```
# 100
```

```
# 60
```

Обход пар

```
prices = {"apple": 100, "banana": 60}
```

```
for name, price in prices.items():  
    print(name, "costs", price)
```

```
# apple costs 100
```

```
# banana costs 60
```

Обновление представлений

```
prices = {"apple": 100, "banana": 60}
names_view = prices.keys()
print(names_view)
# dict_keys(['apple', 'banana'])
```

```
prices["cherry"] = 250
print(names_view)
# dict_keys(['apple', 'banana', 'cherry'])
print(len(names_view))
# 3
```

Изменение при обходе

```
numbers = {"a": 1, "b": 2, "c": 3}
```

```
for key in numbers:  
    if numbers[key] == 1:  
        del numbers[key]
```

```
# RuntimeError: dictionary changed size...
```



Чтение элементов

Квадратные скобки

```
user = {"name": "Alice", "age": 25}
```

```
print(user["name"]) # Alice
```

```
print(user["age"]) # 25
```

Ошибка KeyError

```
user = {"name": "Alice", "age": 25}
```

```
print(user["city"])  
# KeyError: 'city'
```

Метод `get()`

```
user = {"name": "Alice", "age": 25}
```

```
print(user.get("name"))
```

```
# Alice
```

```
print(user.get("name", "unknown"))
```

```
# Alice - ключ есть, default игнорируется
```

```
print(user.get("city"))
```

```
# None - ключа нет, default не указан
```

```
print(user.get("city", "unknown"))
```

```
# unknown - ключа нет, вернули переданный default
```

Практический пример

```
text = "banana"
letters_count = {}

for letter in text:
    letters_count[letter] = (
        letters_count.get(letter, 0) + 1
    )

print(letters_count)
# {'b': 1, 'a': 3, 'n': 2}
```

Метод `setdefault()`

```
user = {"name": "Alice"}

print(user.setdefault("name", "Bob"))
# Alice - ключ был, значение не тронуто
print(user.setdefault("age", 25))
# 25 - ключа не было, он добавлен

print(user)
# {'name': 'Alice', 'age': 25}
```

Практический пример

```
words = ["apple", "banana", "blueberry"]
words_by_letter = {}

for word in words:
    words_by_letter.setdefault(word[0], []).append(word)

print(words_by_letter)
# {'a': ['apple'], 'b': ['banana', 'blueberry']}
```

Что выбрать

ключ обязан быть

`my_dict[key]`

ключ может отсутствовать

`my_dict.get(key, default)`

может отсутствовать и нужен сразу

`my_dict.setdefault(key, default)`



Изменение словарей

Квадратные скобки

```
user = {"name": "Alice", "age": 25}
```

```
user["age"] = 26
```

```
user["city"] = "Paris"
```

```
print(user)
```

```
# {'name': 'Alice', 'age': 26, 'city': 'Paris'}
```

Метод update()

```
user = {"name": "Alice", "age": 25}
```

```
user.update({"age": 26, "city": "Paris"})
```

```
print(user)
```

```
# {'name': 'Alice', 'age': 26, 'city': 'Paris'}
```

```
user.update([("job", "engineer"), ("age", 27)])
```

```
print(user)
```

```
# {'name': 'Alice', 'age': 27, 'city': 'Paris', 'job': 'engineer'}
```

```
user.update(city="Berlin")
```

```
print(user)
```

```
# {'name': 'Alice', 'age': 27, 'city': 'Berlin', 'job': 'engineer'}
```

update() возвращает None

```
user = {"name": "Alice"}  
  
result = user.update({"age": 25})  
  
print(result)  
# None  
print(user)  
# {'name': 'Alice', 'age': 25}
```

Оператор |

```
defaults = {"theme": "light", "font_size": 12}  
custom = {"font_size": 16, "language": "en"}
```

```
merged = defaults | custom
```

```
print(merged)  
# {'theme': 'light', 'font_size': 16, 'language': 'en'}  
print(defaults)  
# {'theme': 'light', 'font_size': 12}  
print(custom)  
# {'font_size': 16, 'language': 'en'}
```

Оператор |=

```
settings = {"theme": "light", "font_size": 12}
```

```
settings |= {"font_size": 16}
```

```
settings |= [("language", "en")]
```

```
print(settings)
```

```
# {'theme': 'light', 'font_size': 16, 'language': 'en'}
```

update, | и |=

update(other)

меняет исходный

возвращает None

first | second

**не меняет
исходный**

новый словарь

first |= second

меняет исходный

тот же словарь



Удаление элементов

Оператор del

```
user = {"name": "Alice", "age": 25, "city": "Paris"}
```

```
del user["city"]  
print(user)  
# {'name': 'Alice', 'age': 25}
```

```
del user["job"]  
# KeyError: 'job'
```

Метод pop()

```
user = {"name": "Alice", "age": 25, "city": "Paris"}
```

```
removed_city = user.pop("city")
```

```
print(removed_city) # Paris
```

```
print(user) # {'name': 'Alice', 'age': 25}
```

```
print(user.pop("job", None))
```

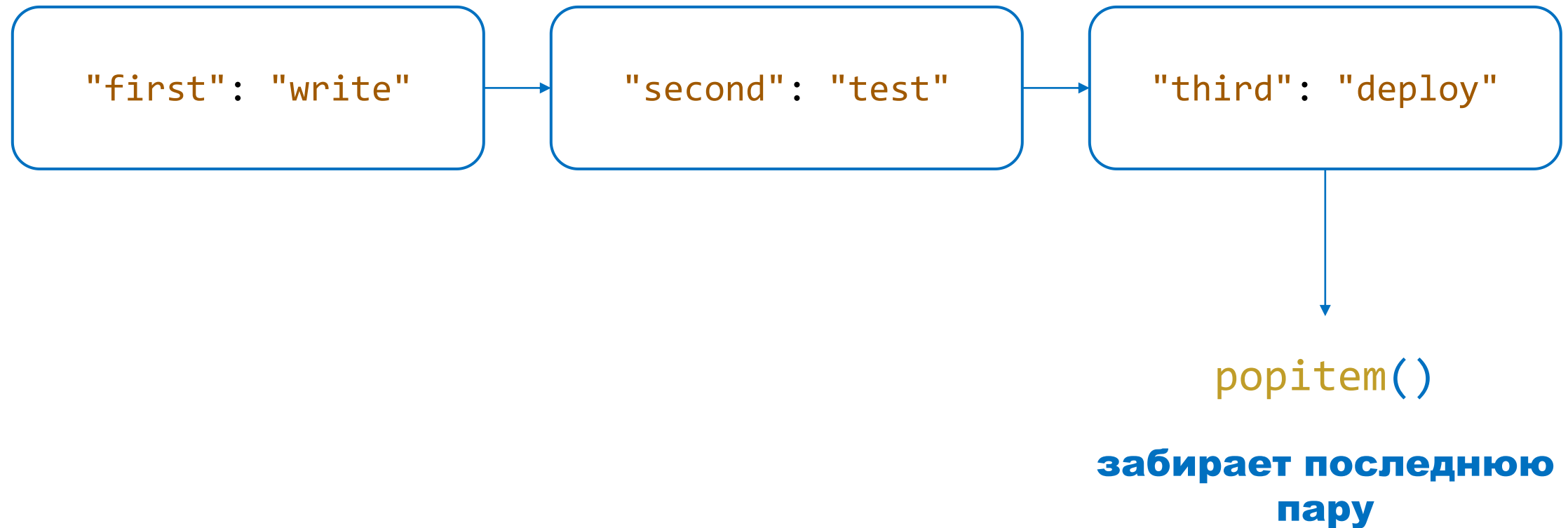
```
# None
```

```
print(user.pop("job"))
```

```
# KeyError: 'job'
```

popitem: с конца

порядок добавления



Метод popitem()

```
user = {"name": "Alice", "age": 25, "city": "Paris"}
```

```
last_item = user.popitem()  
print(last_item)    # ('city', 'Paris')  
print(user)         # {'name': 'Alice', 'age': 25}
```

```
key, value = user.popitem()  
print(key, value)   # age 25
```

```
empty_dict = {}  
empty_dict.popitem()  
# KeyError: 'popitem(): dictionary is empty'
```

Практический пример

```
tasks = {"first": "write", "second": "test", "third": "deploy"}
```

```
while tasks:  
    task_name, task_action = tasks.popitem()  
    print(task_name, task_action)
```

```
# third deploy  
# second test  
# first write
```

Метод clear()

```
user = {"name": "Alice", "age": 25}
same_user = user # вторая ссылка на тот же словарь

user.clear()

print(user)      # {}
print(same_user) # {}
```

Что выбрать

удалить по ключу

```
del my_dict[key]
```

удалить и получить значение

```
my_dict.pop(key)
```

безопасно удалить

```
my_dict.pop(key, None)
```

удалить последнюю пару

```
my_dict.popitem()
```

удалить все пары

```
my_dict.clear()
```



Поиск в словаре

Оператор in

```
user = {"name": "Alice", "age": 25}
```

```
print("name" in user)      # True
```

```
print("city" in user)      # False
```

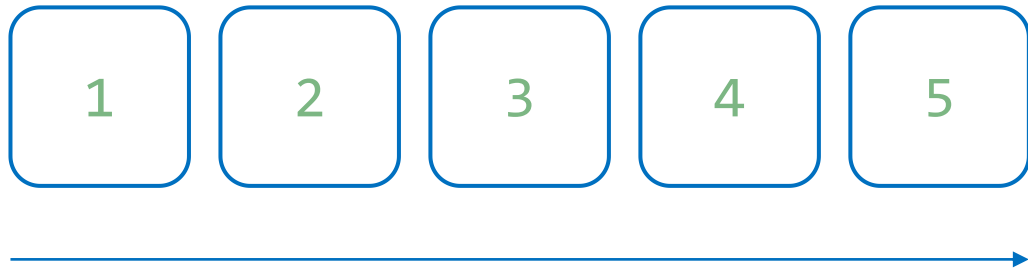
```
print("city" not in user)  # True
```

```
print("Alice" in user)
```

```
# False - "Alice" является значением
```

Список vs словарь

список



перебор по очереди

словарь



сразу к нужному месту

Время поиска

```
from timeit import timeit

search_repeats = 100 # сколько раз повторяем поиск

for items_count in (1_000, 100_000, 1_000_000):
    numbers_list = list(range(items_count))
    numbers_dict = dict.fromkeys(numbers_list)
    missing_number = -1

    list_time = timeit("missing_number in numbers_list", globals=globals(), number=search_repeats)
    dict_time = timeit("missing_number in numbers_dict", globals=globals(), number=search_repeats)

    print(f"{items_count:>9} items | list: {list_time:.6f} s | dict: {dict_time:.6f} s")
```

Вывод

1000 items	list: 0.000711 s	dict: 0.000003 s
100000 items	list: 0.074992 s	dict: 0.000005 s
1000000 items	list: 0.785587 s	dict: 0.000005 s



Множество: обзор

Уникальные элементы



дубликаты исчезают

Удаление дубликатов

```
numbers = [1, 2, 2, 3, 3, 3, 1]
unique_numbers = list(set(numbers))

print(len(unique_numbers))
# 3
```

Уникальные элементы

```
numbers = {1, 2, 2, 3, 3, 3}
```

```
print(numbers)
```

```
# {1, 2, 3} - дубликаты исчезли
```

```
print(len(numbers))
```

```
# 3
```

Характеристики множества

гетерогенное

ограниченное

неупорядоченное

изменяемое

итерируемое

Гетерогенность

```
mixed_set = {  
    42, "hello", 3.14, True, None, (1, 2)  
}
```

```
print(len(mixed_set))  
# 6
```

Хешируемые элементы

```
valid_set = {(1, 2), "a"}  
# хешируемый кортеж, можно
```

```
invalid_set = {[1, 2], "a"}  
# TypeError: unhashable type: 'list'
```

Ограниченность

```
colors = {"red", "green", "blue"}
```

```
print(len(colors))    # 3
```

```
print(len(set()))     # 0
```

Нет индексов

```
colors = {"red", "green", "blue"}
```

```
print(colors[0])
```

```
# TypeError: 'set' object is not subscriptable
```

Изменяемость

```
tags = {"python", "code"}  
identity_before = id(tags)  
  
tags.add("study")  
  
print(len(tags))  
# 3  
print(id(tags) == identity_before)  
# True
```

Итерируемость

```
colors = {"red", "green", "blue"}
```

```
for color in colors:  
    print(color)
```

```
# green
```

```
# red
```

```
# blue
```

dict и set

dict

пары ключ → значение

порядок вставки сохраняется

пустой: {}

set

только элементы

порядок не определен

пустое: set()

Где применяются множества

удаление дубликатов

проверка принадлежности

число уникальных значений

**объединение, пересечение,
разность**



Создание множеств

Способы создания

```
{1, 2, 3}
```

литерал

```
set(iterable)
```

конструктор

```
{item for ...}
```

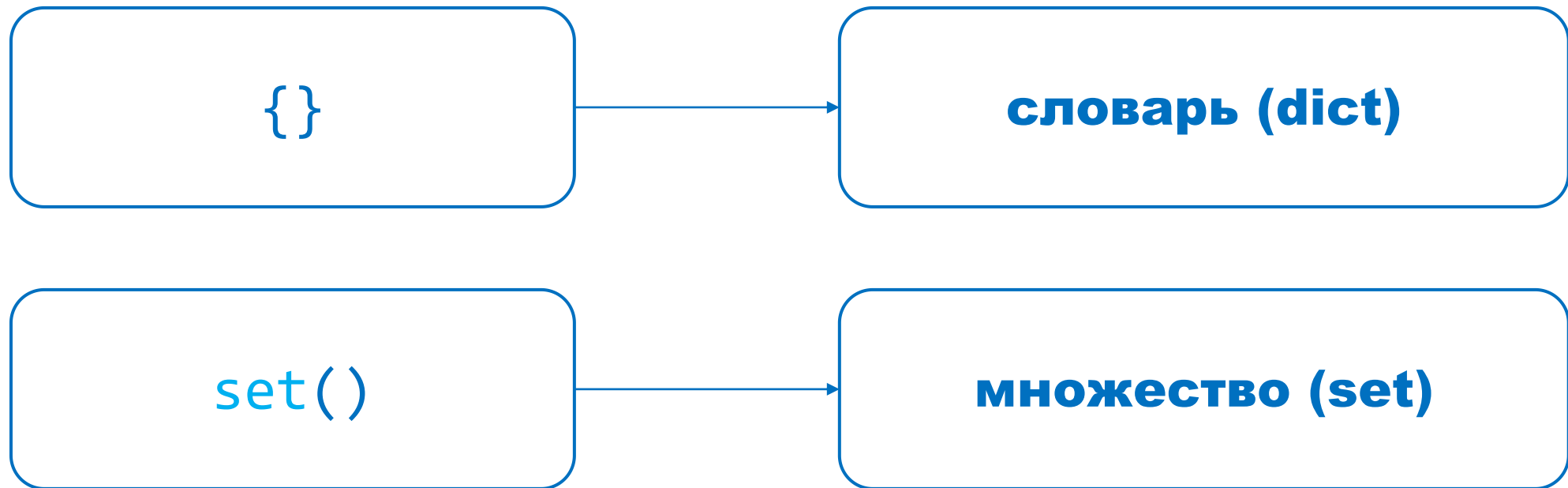
включение

Литералы множеств

```
primes = {2, 3, 5, 7}
letters = {"a", "b", "a", "c"}

print(primes)
# {2, 3, 5, 7}
print(letters)
# {'a', 'b', 'c'}
```

Пустое множество



Пустое множество

```
empty_dict = {}  
empty_set = set()  
  
print(type(empty_dict))  
# <class 'dict'>  
print(type(empty_set))  
# <class 'set'>  
print(empty_set)  
# set()
```

Объект set()

```
numbers_set = set([1, 2, 2, 3])  
print(numbers_set)  
# {1, 2, 3}
```

```
letters_set = set("hello")  
print(letters_set)  
# {'h', 'e', 'l', 'o'}
```

```
keys_set = set({"a": 1, "b": 2})  
print(keys_set)  
# {'a', 'b'}
```

Ошибки `set()`

```
set(1, 2, 3)
```

```
# TypeError: set expected at most 1 argument, got 3
```

```
set(42)
```

```
# TypeError: 'int' object is not iterable
```

```
set([[1, 2], [3, 4]])
```

```
# TypeError: unhashable type: 'list'
```

set() vs литерал

```
print({"hello"})  
# {'hello'}
```

```
print(set("hello"))  
# {'h', 'e', 'l', 'o'}
```

Включение множества

{выражение **for** переменная **in** источник}

```
squares = {number ** 2 for number in range(-3, 4)}
```

```
print(squares)
```

```
# {0, 1, 4, 9}
```

Включение с условием

{выражение **for** переменная **in** источник **if** условие}

```
words = ["apple", "avocado", "banana", "apple"]
```

```
first_letters = {word[0] for word in words}
```

```
long_words = {word for word in words if len(word) > 5}
```

```
print(first_letters)
```

```
# {'a', 'b'}
```

```
print(long_words)
```

```
# {'avocado', 'banana'}
```



Методы множества

Метод add()

```
colors = {"red", "green"}
```

```
colors.add("blue")
```

```
colors.add("red")
```

```
print(sorted(colors))
```

```
# ['blue', 'green', 'red']
```

```
colors.add(["black"])
```

```
# TypeError: unhashable type: 'list'
```

Метод update()

```
numbers = {1, 2, 3}
```

```
numbers.update([3, 4, 5])
```

```
numbers.update({5, 6}, (7, 8))
```

```
print(numbers)
```

```
# {1, 2, 3, 4, 5, 6, 7, 8}
```

add и update

```
letters.add("abc")
```

один элемент
"abc"

```
letters.update("abc")
```

три элемента
"a", "b", "c"

add() n update()

```
letters = set()
```

```
letters.add("abc")
```

```
print(letters)
```

```
# {'abc'}
```

```
letters.update("abc")
```

```
print(sorted(letters))
```

```
# ['a', 'abc', 'b', 'c']
```

Метод `remove()`

```
colors = {"red", "green", "blue"}
```

```
colors.remove("green")  
print(sorted(colors))  
# ['blue', 'red']
```

```
colors.remove("yellow")  
# KeyError: 'yellow'
```

Метод `discard()`

```
colors = {"red", "green", "blue"}
```

```
colors.discard("green")  
colors.discard("yellow")
```

```
print(sorted(colors))  
# ['blue', 'red']
```

Метод pop()

```
numbers = {10, 20, 30}
```

```
removed_number = numbers.pop()  
print(removed_number in {10, 20, 30})  
# True  
print(len(numbers))  
# 2
```

```
set().pop()  
# KeyError: 'pop from an empty set'
```

Практический пример

```
tasks = {"write", "test", "deploy"}
```

```
while tasks:
```

```
    task = tasks.pop()
```

```
    print("Processing:", task)
```

Метод `clear()`

```
numbers = {1, 2, 3}
```

```
numbers.clear()
```

```
print(numbers)
```

```
# set()
```

Сводка методов

метод	что делает	если элемента нет
<code>add(x)</code>	добавляет элемент	добавляет
<code>update(iterable)</code>	добавляет все элементы	добавляет
<code>remove(x)</code>	удаляет элемент	<code>KeyError</code>
<code>discard(x)</code>	удаляет элемент	—
<code>pop()</code>	удаляет произвольный	<code>KeyError</code> (пустой set)
<code>clear()</code>	удаляет все	—

Оператор in

```
colors = {"red", "green", "blue"}
```

```
print("red" in colors)           # True  
print("yellow" in colors)       # False  
print("yellow" not in colors)   # True
```

```
print([1, 2] in {1, 2, 3})  
# TypeError: unhashable type: 'list'
```



Спасибо за внимание!